

1. What is a Decorator?

- A decorator is a function that adds or changes the behavior of another function without changing its original code.

Original Function

↓

Decorator

↓

Modified / Extended Function

2. Why Do We Use Decorators?

- Suppose we have

```
def add(a, b):  
    return a + b
```

we want to add:

- logging
- authentication
- execution time calculation
- permission checking
- validation

could modify add() every time.

- But decorators allow you to add these features without changing the original function .

3. Function as an Argument

```
def hello():  
    print("Hello")
```

```
def execute(func):  
    func()  
  
execute(hello)
```

4. Returning a Function

```
def outer():  
  
    def inner():  
        print("Hello")  
  
    return inner  
  
x = outer()  
  
x()
```

5. Decorator Example

```
def my_decorator(func):  
  
    def wrapper():  
        print("Before function")  
  
        func()  
  
        print("After function")  
    return wrapper  
  
@my_decorator  
def hello():  
    print("Hello")  
  
hello()
```